

UNICAMP - Universidade Estadual de Campinas

FEEC - Faculdade de Engenharia Elétrica e de Computação

Câmbio CVT (v2) com BitDogLab V7 RELATÓRIO DE PROJETO

Link deste documento (acesso Comentador para UNICAMP): [Link](#)
[Repositório no GitHub](#)
[Vídeo de demonstração](#)

Arthur Lucas Da Silva Nogueira
Raul Galdino Tancredo

RA: 213293
RA: 223908

Prof. Eric Rohmer

Campinas
2026

1. Descrição do Sistema

O sistema desenvolvido consiste na implementação de um Câmbio de Transmissão Continuamente Variável (CVT) simulado e controlado remotamente via interface sem fio. O projeto utiliza a plataforma BitDogLab V7 como unidade central de processamento, integrando comunicação serial, controle de potência e telemetria.

Utilizando uma ponte H L293D, o controle dos motores é dividido em dois eixos de atuação:

1. Eixo motor: motor de corrente contínua (DC) responsável pela força motriz principal, conectado ao cone principal.
2. Atuador do câmbio: motor de corrente contínua acoplado a um sistema de fuso (atuador linear), responsável por alterar a posição física da correia do câmbio CVT. No sistema de fuso, há a polia (que tem o anel o-ring/elástico girando livremente) e a guia, que faz com que o movimento de rotação do fuso se transforme em movimento linear.

A interface homem-máquina (IHM) é realizada de forma remota através de um módulo Bluetooth (conectado à interface UART da placa) com a integração de um aplicativo no celular (“Arduino Bluetooth Control” na Play Store do Google). O display OLED da placa é utilizado para exibir a telemetria em tempo real.

Para termos um controle de posicionamento do câmbio, foi incluído o sensor *Time of Flight* (ToF) VL6180X da STM. Com este sensor acoplado ao câmbio linear, temos um feedback de posicionamento para controlar a velocidade de saída do câmbio.

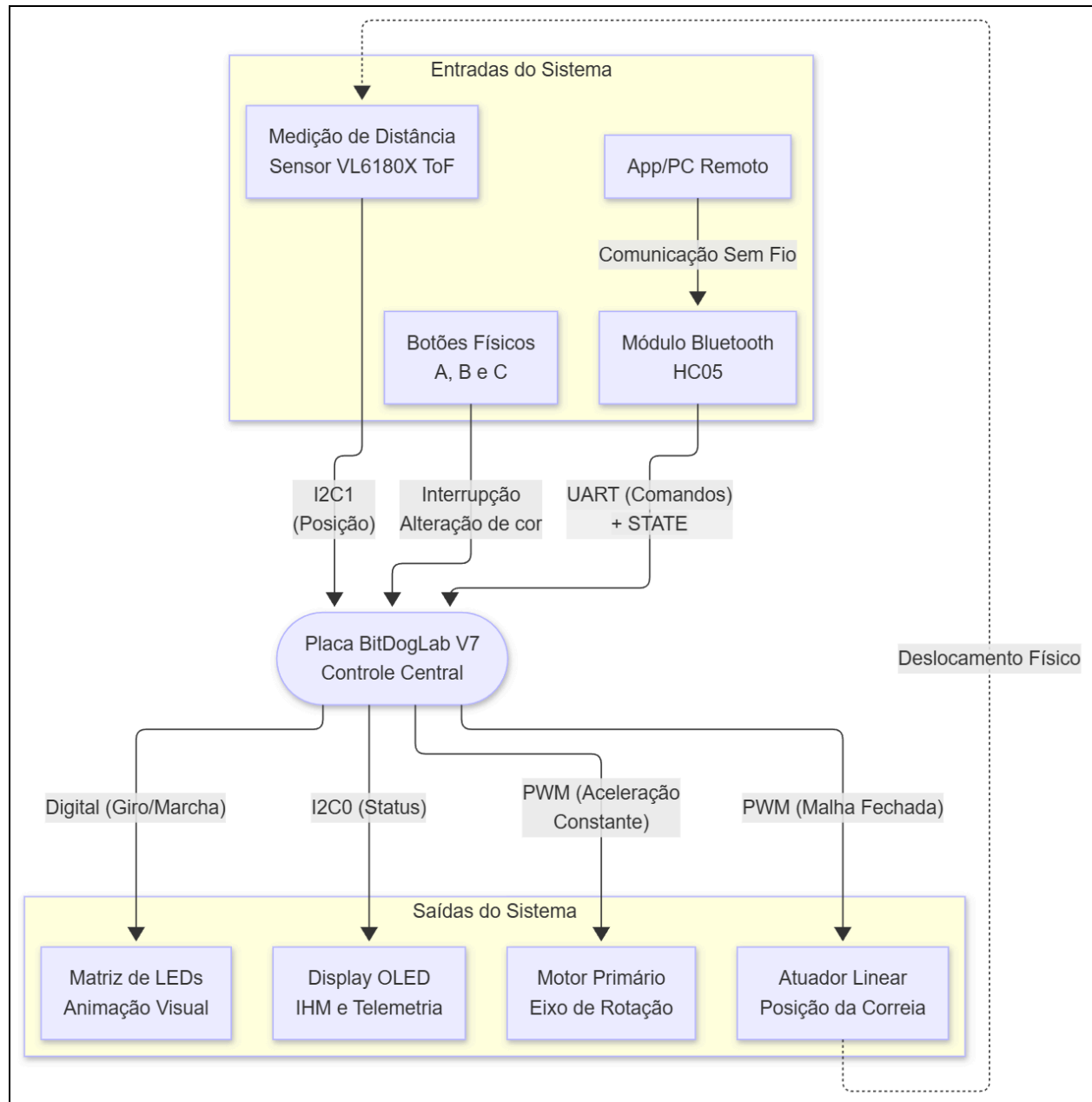


Figura 1 - Arquitetura do sistema.

2. Código fonte

- Códigos de configuração e teste:
 - Configuração módulo Bluetooth: [libs/tools/hc05_atMode_config.py](#)
 - Mapeamento de comandos Bluetooth: [libs/tools/hc05_bluetooth_mapping.py](#)
 - Calibração do VL6180x: [libs/tools/vl6180x_calibrate.py](#)
 - Teste do VL6180x: [libs/tools/vl6180x_test.py](#)

- Biblioteca do sensor VL6180X: [libs/vl6180x.py](https://github.com/Ledbelly2142/VL6180X/blob/master/vl6180x.py)

Python

```
"""
GitHub: https://github.com/Ledbelly2142/VL6180X/blob/master/vl6180x.py
Author: Ledbelly2142 (https://github.com/Ledbelly2142)
"""

import ustruct
import struct
import time
from machine import I2C, Pin

i2c_tof = I2C(0, sda=Pin(8), scl=Pin(9))
offsetVal = 35

class Sensor:
    def __init__(self, i2c, address=0x29):
        self.i2c_tof = i2c
        self._address = address
        self.default_settings()
        self.init()

    def myWrite16(self, register, regValue):
        """ write a byte to specified 16 bit register """
        return self.i2c_tof.writeto_mem(self._address, register,
        bytearray([regValue]), addrsize=16), 'big'

    def myRead16(self, register):
        """read 1 bit from 16 byte register"""
        value = int.from_bytes(
            self.i2c_tof.readfrom_mem(self._address, register, 1, addrsize=16),
            'big'
        )
        return value & 0xFFFF

    def init(self):
        if self.myRead16(0x0016) != 1:
            raise RuntimeError("Failure reset")

        # Recommended setup from the datasheet
        self.myWrite16(0x0207, 0x01)
        self.myWrite16(0x0208, 0x01)
        self.myWrite16(0x0096, 0x00)
        self.myWrite16(0x0097, 0xfd)
        self.myWrite16(0x00e3, 0x00)
        self.myWrite16(0x00e4, 0x04)
        self.myWrite16(0x00e5, 0x02)
        self.myWrite16(0x00e6, 0x01)
        self.myWrite16(0x00e7, 0x03)
        self.myWrite16(0x00f5, 0x02)
        self.myWrite16(0x00d9, 0x05)
        self.myWrite16(0x00db, 0xce)
```

```

self.myWrite16(0x00dc, 0x03)
self.myWrite16(0x00dd, 0xf8)
self.myWrite16(0x009f, 0x00)
self.myWrite16(0x00a3, 0x3c)
self.myWrite16(0x00b7, 0x00)
self.myWrite16(0x00bb, 0x3c)
self.myWrite16(0x00b2, 0x09)
self.myWrite16(0x00ca, 0x09)
self.myWrite16(0x0198, 0x01)
self.myWrite16(0x01b0, 0x17)
self.myWrite16(0x01ad, 0x00)
self.myWrite16(0x00ff, 0x05)
self.myWrite16(0x0100, 0x05)
self.myWrite16(0x0199, 0x05)
self.myWrite16(0x01a6, 0x1b)
self.myWrite16(0x01ac, 0x3e)
self.myWrite16(0x01a7, 0x1f)
self.myWrite16(0x0030, 0x00)

def default_settings(self):
    self.myWrite16(0x0011, 0x10)
    self.myWrite16(0x010A, 0x30)
    self.myWrite16(0x003f, 0x46)
    self.myWrite16(0x0031, 0xFF)
    self.myWrite16(0x0040, 0x63)
    self.myWrite16(0x002E, 0x01)
    self.myWrite16(0x0024, 100)
    self.myWrite16(0x001B, 0x09)
    self.myWrite16(0x003E, 0x0A)
    self.myWrite16(0x0014, 0x24)
    self.myWrite16(0x002D, 0x10)
    self.myWrite16(0x0022, 0x28)
    self.myWrite16(0x001C, 0x3F)
    self.myWrite16(0x0120, 0x01)

def identify(self):
    return {
        'model': self.myRead16(0x0000),
        'revision': (self.myRead16(0x0001), self.myRead16(0x0002)),
        'module_revision': (self.myRead16(0x0003), self.myRead16(0x0004)),
        'date': self.myRead16(0x0006),
        'time': self.myRead16(0x0008),
    }

def address(self, address=None):
    if address is None:
        return self._address
    if not 8 <= address <= 127:
        raise ValueError("Wrong address")
    self.myWrite16(0x0212, address)
    self._address = address

```

```
def range(self):
    """Previne falhas elétricas do hardware (I2C OSError) e retorna None de
    forma segura"""
    try:
        self.myWrite16(0x0018, 0x01) # Sysrange start

        # Timeout simples para evitar travamento em loops infinitos caso o I2C
        congele
        timeout = 50
        while (self.myRead16(0x004F) & 0x04) == 0:
            time.sleep(0.005)
            timeout -= 1
            if timeout <= 0:
                print("Aviso: Timeout de leitura do sensor ToF.")
                return None

        status = self.myRead16(0x004D) >> 4
        distancia_crua = self.myRead16(0x0062)
        self.myWrite16(0x0015, 0x07) # Clear Interrupt

        if status != 0:
            print(f"Erro interno do ToF! Código: {status}")
            return None

        distancia_real = distancia_crua
        if distancia_real < 0:
            distancia_real = 0

        return (0 if ((distancia_real - offsetVal) < 0) else (distancia_real -
        offsetVal))

    except OSError:
        # Captura o erro EIO [Errno 5] físico sem derrubar o programa
        principal
        print("Alerta: Falha física instantânea de comunicação I2C com o
        ToF.")
        return None
```

- Setup do projeto: [libs/setup.py](#)

Python

```
"""
    arquivo de importação de bibliotecas, configuração de interfaces e variáveis
    """

from machine import Pin, ADC, I2C, PWM
import neopixel, utime, machine, time
import ssd1306, vl6180x

# 1. Variáveis Globais e Cores
```

```

corVermelha = (20, 0, 0)
corAzul = (0, 0, 20)
corVerde = (0, 20, 0)
cor_travada = corVermelha # Motor começa vermelho
ultimo_tempo_btn = 0
ultimo_tempo_pisca = utime.ticks_ms()
indice_frame = 0

# 2. Variáveis de Controle do Motor Virtual
CENTRO_JOYSTICK = 32768
ZONA_MORTA = 4000
MAX_LEDS_POR_SEGUNDO = 100 # Ajuste a velocidade máxima aqui

# 3. Configuração dos Botões (A, B e C)
botao_a = Pin(5, Pin.IN, Pin.PULL_UP)
botao_b = Pin(6, Pin.IN, Pin.PULL_UP)
botao_c = Pin(10, Pin.IN, Pin.PULL_UP)

# 4. Configuração do Joystick
joystick_x = ADC(Pin(27))
joystick_y = ADC(Pin(26))
joystick_sw = Pin(22, Pin.IN, Pin.PULL_UP)

# 5. Configuração do Display OLED (SSD1306)
# Usando os pinos I2C0 da BitDogLab V7 (SDA=2, SCL=3)
i2c = I2C(1, scl=Pin(3), sda=Pin(2), freq=400000)
display = ssd1306.SSD1306_I2C(128, 64, i2c, addr=0x3C)
display.fill(0)
display.show()
ATT_DISPLAY_MS = 100

# 6. Configuração da Matriz de LEDs
NUM_LEDS = 25
np = neopixel.NeoPixel(Pin(7), NUM_LEDS)
LED_MEIO = 12
# Buffer circular da animação do motor
FRAMES_MOTOR = [[14, 13], [15, 16], [16, 24], [16, 23], [17, 22],
                 [18, 21], [18, 20], [18, 19], [10, 11], [8, 9],
                 [0, 8], [1, 8], [2, 7], [3, 6], [4, 6], [5, 6] ]

# 7. Configuração dos controles de Motor (Ponte H) e PWM da placa
# IMPORTANTE: GPIO 0 e 1 são do Bluetooth. Vamos usar os pinos livres do conector IDC.

# Motor Principal (Eixo) - Assumindo ligação no lado Esquerdo do L293D (Pinos 1, 2 e 7)
Mprincipal_DIR1 = Pin(16, Pin.OUT)
Mprincipal_DIR2 = Pin(17, Pin.OUT)
Mprincipal_PWM = PWM(Pin(18)) # GP28 vai no pino 1 (EN1,2) do L293D
Mprincipal_PWM.freq(1000)
Mprincipal_PWM.duty_u16(0)
TEMPO_RAMPA_MS = 1000

```

```
# Motor Câmbio (Atuador CVT) - EXATAMENTE COMO NO SEU ESQUEMÁTICO (Lado Direito)
Mcambio_DIR1 = Pin(19, Pin.OUT)      # GP18 (Fio Roxo) vai no IN3
Mcambio_DIR2 = Pin(20, Pin.OUT)      # GP20 (Fio Branco) vai no IN4
Mcambio_PWM = PWM(Pin(28))           # GP19 (Fio Cinza) vai no EN3,4 (Enable/PWM)
Mcambio_PWM.freq(1000)
Mcambio_PWM.duty_u16(0)
TEMPO_PASSO_CVT_MS = 5000 # 2000 ms = 2 segundos

# 8. Configura o LED Azul da placa (GPIO 12) para feedback visual
led_azul = machine.Pin(12, machine.Pin.OUT)
led_azul.value(0) # Inicia desligado

# 9. Inicializa a UART0 para o Bluetooth (Pinos do Conector J2)
bluetooth = machine.UART(0, baudrate=9600, tx=machine.Pin(0), rx=machine.Pin(1))
status_bt = machine.Pin(4, machine.Pin.IN)      # Pino que vai ler o pino 'STATE'
do módulo Bluetooth

# 10. Mapeamento das entradas bluetooth
outButtonDir = '0'
frontDir = 'F'
backDir = 'B'
rightDir = 'R'
leftDir = 'L'
triangleDir = 'T'
circleDir = 'C'
squareDir = 'S'
crossDir = 'X'
startDir = 'A'
pauseDir = 'P'

# 11. Inicializa o sensor
#i2c_tof = I2C(0, sda=Pin(8), scl=Pin(9)) #esta declaracao já está na biblioteca
do sensor ToF
i2c_tof = vl6180x.i2c_tof
sensor_tof = vl6180x.Sensor(i2c_tof) # type: ignore
```

- Arquivo principal: link

Python

```
from setup import *
```

```
# VARIÁVEIS PARA O CVT ASSÍNCRONO BASEADO EM DISTÂNCIA
cvt_em_movimento = False
direcao_cvt = 0      # 1 avança (diminui mm), -1 recua (aumenta mm)
alvo_tof_mm = 0.0    # Distância que queremos atingir
janela_leituras = [] # type: list[float]
TAMANHO_JANELA = 30  # Média das últimas 30 leituras
PASSO_MM = 4.0       # Quantos mm o câmbio anda por vez
MAX_DIST_MM = 24.0   # Fundo do câmbio (0%)
```



```

MIN_DIST_MM = 1.0      # Batente do câmbio (100%)

# =====
# FUNÇÕES DE HARDWARE (MOTORES, TELA E SENSOR)
# =====

# MÉTODO DE LEITURA E SINCRONIZAÇÃO DO SENSOR TOF
def sincronizar_posicao_cvt():
    global posicao_cvt
    print("Sincronizando posição atual do sensor ToF...")
    leituras_locais = []

    # Coleta 10 amostras rápidas para ignorar ruídos de leitura única
    for _ in range(10):
        leitura = sensor_tof.range()
        if leitura is not None and leitura > 0:
            leituras_locais.append(leitura)
            utime.sleep(0.01)

    if len(leituras_locais) > 0:
        distancia_calculada = sum(leituras_locais) / len(leituras_locais)
    else:
        # Se falhar totalmente por erro I2C, assume a última posição segura
        conhecida
        distancia_calculada = MAX_DIST_MM

    # Trava dentro dos limites físicos do projeto
    distancia_calculada = max(MIN_DIST_MM, min(MAX_DIST_MM, distancia_calculada))

    # 1. Atualiza a variável global de porcentagem
    porcentagem_calculada = ((MAX_DIST_MM - distancia_calculada) / (MAX_DIST_MM -
MIN_DIST_MM)) * 100
    posicao_cvt = max(0, min(100, int(porcentagem_calculada)))

    print(f"Posição Sincronizada: {distancia_calculada:.1f} mm ({posicao_cvt}%)")
    return distancia_calculada

# Lida com as informações do display OLED
def printOled(velocidade, direcao, posicao_cvt, bt_conectado):
    texto_sentido = "Horario" if direcao == 1 else "Anti-Hor."
    texto_sentido = "Parado" if velocidade == 0 else texto_sentido
    texto_bt = "ON" if bt_conectado == 1 else "OFF"
    display.fill(0)
    display.text("Cambio CVT", 20, 0)
    display.text("-" * 16, 0, 10)
    display.text(f"Motor: {texto_sentido}", 0, 25)
    display.text(f"Pos. CVT: {posicao_cvt}%", 0, 40)
    display.text(f"Bluetooth: {texto_bt}", 0, 55)
    display.show()

# Controle do motor principal

```

```
def controlar_motor_principal(velocidade_percentual, direcao, fazer_rampa=False):
    if fazer_rampa and velocidade_percentual > 0:
        passos = 10
        tempo_por_passo = (TEMPO_RAMPA_MS / 2000) / passos
        passo_pwm = max(1, velocidade_percentual // passos)
        for v in range(velocidade_percentual, -1, -passo_pwm):
            Mprincipal_PWM.duty_u16(int((v / 100) * 65535))
            utime.sleep(tempo_por_passo)
        Mprincipal_DIR1.value(0)
        Mprincipal_DIR2.value(0)
        Mprincipal_PWM.duty_u16(0)
        utime.sleep(0.05)
        if direcao == 1:
            Mprincipal_DIR1.value(1)
            Mprincipal_DIR2.value(0)
        else:
            Mprincipal_DIR1.value(0)
            Mprincipal_DIR2.value(1)
        for v in range(0, velocidade_percentual + 1, passo_pwm):
            Mprincipal_PWM.duty_u16(int((v / 100) * 65535))
            utime.sleep(tempo_por_passo)
        Mprincipal_PWM.duty_u16(int((velocidade_percentual / 100) * 65535))
    else:
        if velocidade_percentual > 0:
            if direcao == 1:
                Mprincipal_DIR1.value(1)
                Mprincipal_DIR2.value(0)
            else:
                Mprincipal_DIR1.value(0)
                Mprincipal_DIR2.value(1)
        else:
            Mprincipal_DIR1.value(0)
            Mprincipal_DIR2.value(0)
        Mprincipal_PWM.duty_u16(int((velocidade_percentual / 100) * 65535))

# Movimentação do câmbio, velocidade sempre em 100%
def iniciar_movimento_cvt(direcao):
    if direcao == 1:
        Mcambio_DIR1.value(1)
        Mcambio_DIR2.value(0)
    elif direcao == -1:
        Mcambio_DIR1.value(0)
        Mcambio_DIR2.value(1)
    else:
        return
    Mcambio_PWM.duty_u16(65535)

# VARIÁVEIS DE CONTROLE DOS BOTÕES FÍSICOS
flag_homing = False
flag_cancelar = False

# =====
```

```
# Função de Interrupção dos Botões
# =====
def trata_interrupcao_botao(pino):
    global cor_travada, ultimo_tempo_btn, flag_homing, flag_cancelar
    tempo_atual = utime.ticks_ms()

    if utime.ticks_diff(tempo_atual, ultimo_tempo_btn) > 200:
        if pino == botao_a:
            cor_travada = corVermelha
            flag_homing = True      # Solicita o Homing (Voltar a 0%)
        elif pino == botao_b:
            cor_travada = corAzul
        elif pino == botao_c:
            cor_travada = corVerde
            flag_cancelar = True    # Solicita a Parada Total (E-Stop)

        ultimo_tempo_btn = tempo_atual

botao_a.irq(trigger=Pin.IRQ_FALLING, handler=trata_interrupcao_botao)
botao_b.irq(trigger=Pin.IRQ_FALLING, handler=trata_interrupcao_botao)
botao_c.irq(trigger=Pin.IRQ_FALLING, handler=trata_interrupcao_botao)

# =====
# INICIALIZAÇÃO E LOOP PRINCIPAL
# =====
ultimo_tempo_pisca = utime.ticks_ms()
ultimo_tempo_oled = utime.ticks_ms()

velocidade_atual = 0
direcao_motor = 1

# --- CHAMADA DO MÉTODO ANTES DO WHILE (POSIÇÃO INICIAL) ---
distancia_inicial = sincronizar_posicao_cvt()
# -----

cvt_em_movimento = False
ultimo_tempo_cvt = 0

for i in range(NUM_LEDS):
    np[i] = (0, 0, 0)
np.write()

print("Aguardando comandos Bluetooth...")

while True:
    tempo_atual = utime.ticks_ms()

    # =====
    # AÇÃO DE EMERGÊNCIA / CANCELAR (BOTÃO C)
    # =====
    if flag_cancelar:
        flag_cancelar = False
```

```

    velocidade_atual = 0
    controlar_motor_principal(0, direcao_motor)

    Mcambio_DIR1.value(0)
    Mcambio_DIR2.value(0)
    Mcambio_PWM.duty_u16(0)
    cvt_em_movimento = False

    print("BOTÃO C: Parada de Emergencia Acionada!")

# =====
# AÇÃO DE HOMING / RETORNO À BASE (BOTÃO A)
# =====
if flag_homing:
    flag_homing = False
    if not cvt_em_movimento:
        alvo_tof_mm = MAX_DIST_MM
        direcao_cvt = -1

        # --- CHAMADA DO MÉTODO DURANTE O FUNCIONAMENTO ---
        leitura_estavel = sincronizar_posicao_cvt()

        # Inicializa a janela com o valor atualizado e limpo
        janela_leituras = [leitura_estavel] * TAMANHO_JANELA

        iniciar_movimento_cvt(direcao_cvt)
        cvt_em_movimento = True
        print(f"BOTÃO A: Homing iniciado a partir de {leitura_estavel:.1f}
mm...")

# 1. ATUALIZAÇÃO DO DISPLAY OLED
if utime.ticks_diff(tempo_atual, ultimo_tempo_oled) >= ATT_DISPLAY_MS:
    estado_conexao = status_bt.value()
    if estado_conexao == 0 and velocidade_atual > 0:
        passos = 10
        tempo_por_passo = (TEMPO_RAMPA_MS / 2000) / passos
        passo_pwm = max(1, velocidade_atual // passos)
        for v in range(velocidade_atual, -1, -passo_pwm):
            Mprincipal_PWM.duty_u16(int((v / 100) * 65535))
            utime.sleep(tempo_por_passo)
        velocidade_atual = 0
        controlar_motor_principal(0, direcao_motor)
        printOled(velocidade_atual, direcao_motor, posicao_cvt, estado_conexao)
        ultimo_tempo_oled = tempo_atual

# 2. LEITURA DE COMANDOS BLUETOOTH
if bluetooth.any():
    try:
        dados_brutos = bluetooth.read()
        if dados_brutos is not None:
            comando = dados_brutos.decode('utf-8').strip().upper()

```

```

if comando:
    led_azul.value(1)

    if comando == startDir:
        velocidade_atual = 100
        controlar_motor_principal(velocidade_atual, direcao_motor)

    elif comando == frontDir:
        deve_fazer_rampa = (direcao_motor == -1 and
        velocidade_atual > 0)
        direcao_motor = 1
        controlar_motor_principal(velocidade_atual, direcao_motor,
        deve_fazer_rampa)

    elif comando == backDir:
        deve_fazer_rampa = (direcao_motor == 1 and
        velocidade_atual > 0)
        direcao_motor = -1
        controlar_motor_principal(velocidade_atual, direcao_motor,
        deve_fazer_rampa)

    elif comando == triangleDir:    # Sobe marcha (Avança /
    Diminui mm)
        if not cvt_em_movimento:
            leitura_atual = sensor_tof.range()
            if leitura_atual is not None:
                if leitura_atual > MIN_DIST_MM:
                    alvo_tof_mm = max(MIN_DIST_MM, leitura_atual -
                    PASSO_MM)

                    direcao_cvt = 1
                    iniciar_movimento_cvt(direcao_cvt)
                    cvt_em_movimento = True
                    janela_leituras = [leitura_atual] *
                    TAMANHO_JANELA

        elif comando == crossDir:    # Desce marcha (Recua /
        Aumenta mm)
            if not cvt_em_movimento:
                leitura_atual = sensor_tof.range()
                if leitura_atual is not None:
                    if leitura_atual < MAX_DIST_MM:
                        alvo_tof_mm = min(MAX_DIST_MM, leitura_atual +
                        PASSO_MM)

                        direcao_cvt = -1
                        iniciar_movimento_cvt(direcao_cvt)
                        cvt_em_movimento = True
                        janela_leituras = [leitura_atual] *
                        TAMANHO_JANELA

    elif comando == pauseDir:
        velocidade_atual = 0
        controlar_motor_principal(0, direcao_motor)

```

```

        Mcambio_DIR1.value(0)
        Mcambio_DIR2.value(0)
        Mcambio_PWM.duty_u16(0)
        cvt_em_movimento = False

        led_azul.value(0)
    except UnicodeError:
        pass

# 3. VERIFICADOR ASSÍNCRONO DO MOTOR CVT (MÉDIA MÓVEL COM TRAVA PARA 0MM)
if cvt_em_movimento:
    nova_leitura = sensor_tof.range()
    chegou_no_alvo = False

    # Se houver erro I2C (None) em movimento, assume o último dado estável
    para não travar o loop
    if nova_leitura is None:
        nova_leitura = janela_leituras[-1] if len(janela_leituras) > 0 else
MAX_DIST_MM

    # --- FILTRO INTELIGENTE E ANTITRAVAMENTO ---
    if nova_leitura == 0:
        if direcao_cvt == 1 and len(janela_leituras) > 0 and
janela_leituras[-1] <= (MIN_DIST_MM + 3.0):
            chegou_no_alvo = True
            nova_leitura = MIN_DIST_MM
            print("Seguranca: Sensor detectou colisao/fim de curso em 0mm!")
        else:
            nova_leitura = janela_leituras[-1] if len(janela_leituras) > 0
else MAX_DIST_MM
        elif nova_leitura > 50:
            nova_leitura = janela_leituras[-1] if len(janela_leituras) > 0 else
MAX_DIST_MM

        janela_leituras.pop(0)
        janela_leituras.append(nova_leitura)

        media_atual = sum(janela_leituras) / TAMANHO_JANELA

        porcentagem = ((MAX_DIST_MM - media_atual) / (MAX_DIST_MM - MIN_DIST_MM))
* 100
        posicao_cvt = max(0, min(100, int(porcentagem)))

# 1. Validação de Alvo
if direcao_cvt == 1 and media_atual <= alvo_tof_mm:
    chegou_no_alvo = True
elif direcao_cvt == -1 and media_atual >= (alvo_tof_mm - 0.5):
    chegou_no_alvo = True

# 2. Trava de Segurança dos Batentes Físicos (Baseado na Média)
if media_atual <= MIN_DIST_MM and direcao_cvt == 1:
    chegou_no_alvo = True

```

```

        print("Seguranca: Batente Minimo atingido pela media!")
    elif media_atual >= MAX_DIST_MM and direcao_cvt == -1:
        chegou_no_alvo = True
        print("Seguranca: Batente Maximo atingido pela media!")

# 3. Execução Correta da Parada
if chegou_no_alvo:
    Mcambio_DIR1.value(0)
    Mcambio_DIR2.value(0)
    Mcambio_PWM.duty_u16(0)
    cvt_em_movimento = False

    if direcao_cvt == -1 and media_atual >= (MAX_DIST_MM - 1.0):
        posicao_cvt = 0
        print(f"Cambio Parado. Posicao Final: {posicao_cvt}%
        ({media_atual:.1f} mm)")

    utime.sleep(0.01)

```

3. Desenvolvimento de Projeto

a) CAD 3D - Projeto mecânico

Aplicando algumas melhorias em relação ao projeto 2, foi feita uma nova estrutura com o auxílio de um software CAD. Paredes mais espessas, mudança nas posições dos motores, revisão da posição do eixo do câmbio, etc.

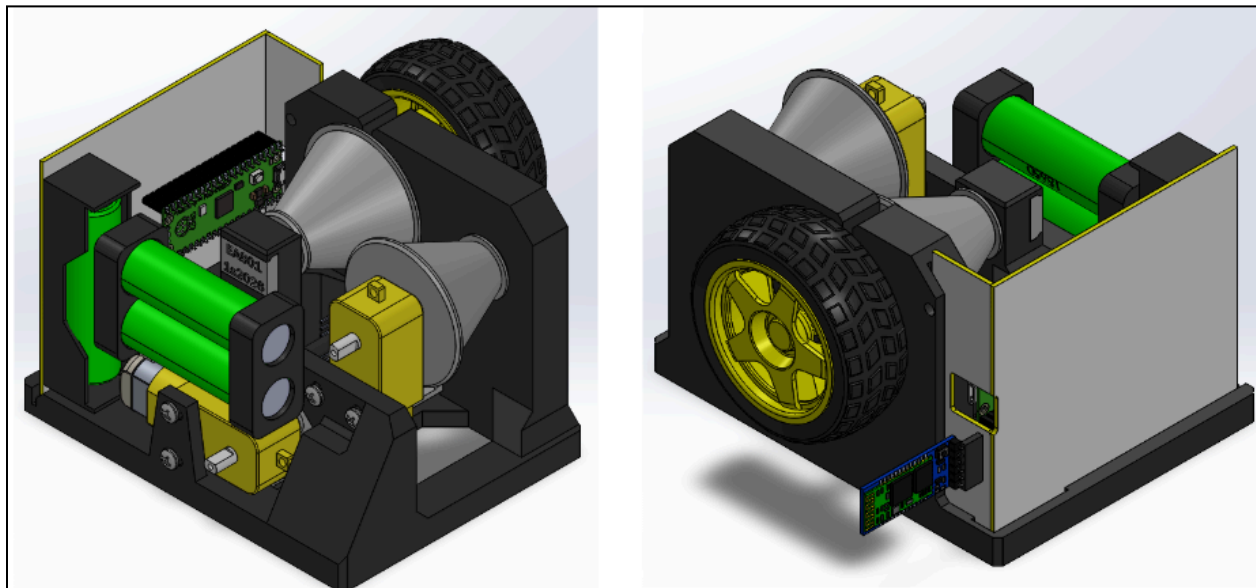


Figura 2 - Projeto CAD (versão 2).

b) CAD 2D - Projeto eletrônico

Utilizando um software para design de circuitos eletrônicos, foi feito o esquemático da placa com as ligações necessárias. O objetivo era fazer a o design da PCB e, com o auxílio da FEEC, fazer as placas conforme projeto. Porém, com alguns acontecimentos na Universidade, foi decidido apenas continuar com uma PCB com os itens soldados, sem termos uma placa dedicada.

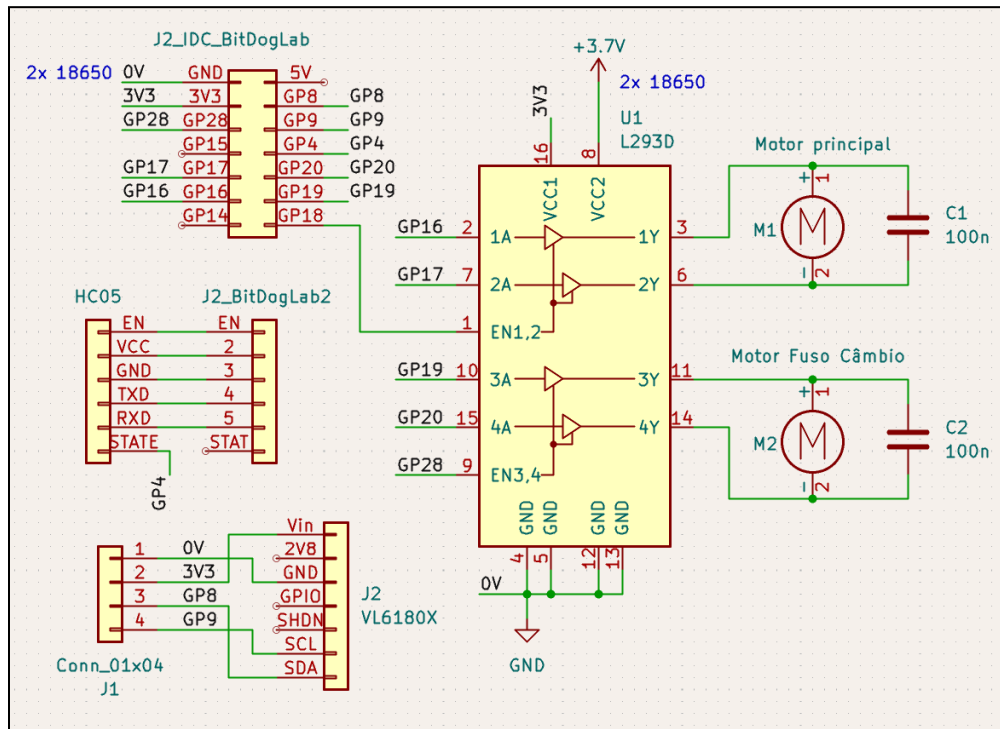


Figura 3 - Projeto esquemático da placa *Shield*.

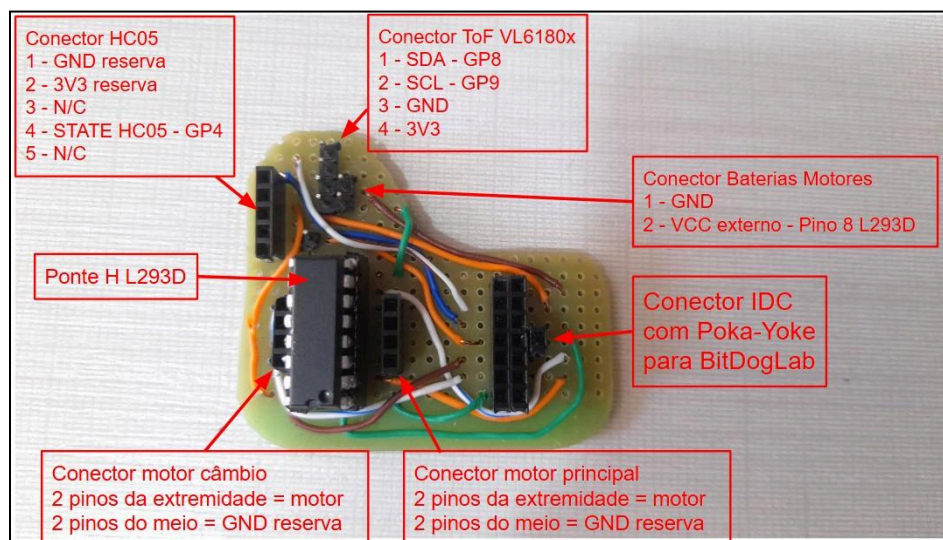


Figura 4 - Placa fabricada.

4. Montagem e desvios de projeto

Com os arquivos .stl, as peças foram impressas e o equipamento foi montado. Como desvios de projeto, foi adicionado um parafuso ao eixo do motor principal para diminuir atrito no giro, as superfícies dos cones tiveram pedaços de lixas colados para aumentar o atrito entre o elástico e o cone e a superfície refletora do ToF foi feita com um pedaço de papel cartão branco pois o plástico utilizado na estrutura é da cor preta, o que diminui a eficácia das leituras do ToF.

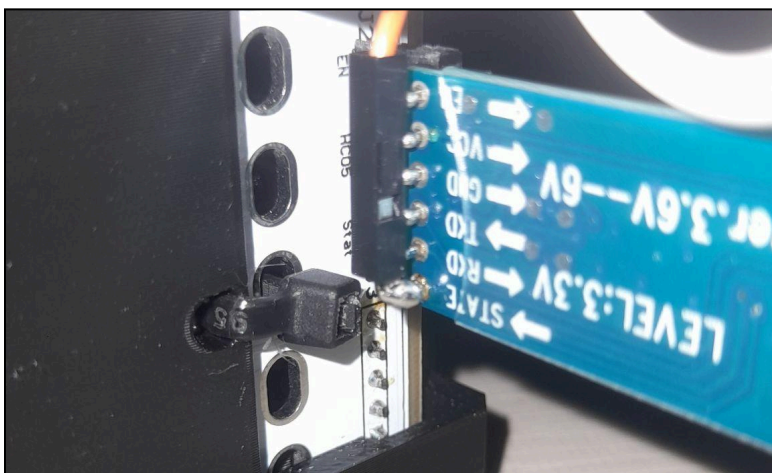


Figura 5 - Jumper soldado no pino de state para ser conectado na shield (STATE→GP4).

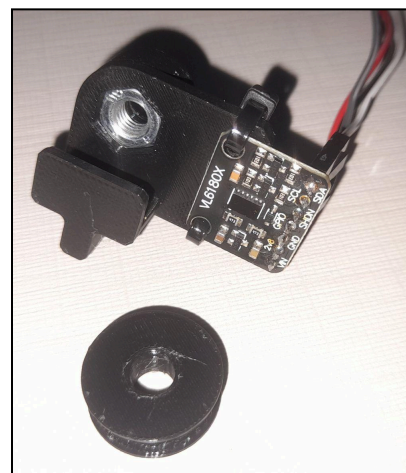


Figura 6 - Sistema de guia linear do eixo, com sensor ToF.

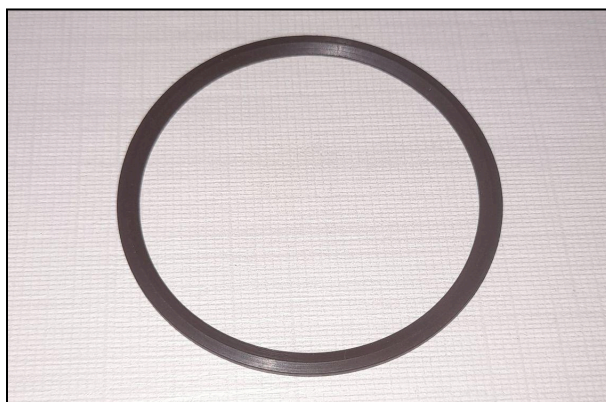


Figura 7 - Anel de vedação utilizado como correia.

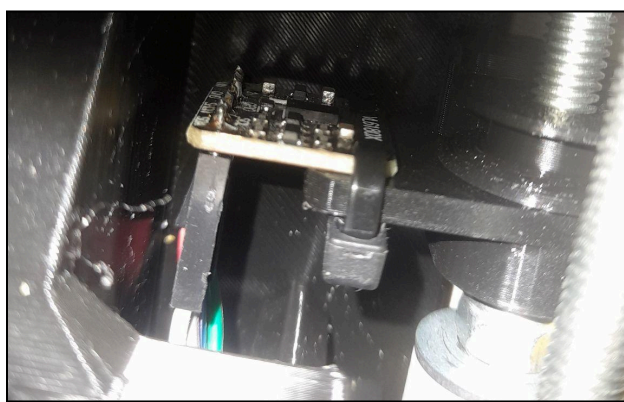


Figura 8 - Guia linear montada no eixo do câmbio.

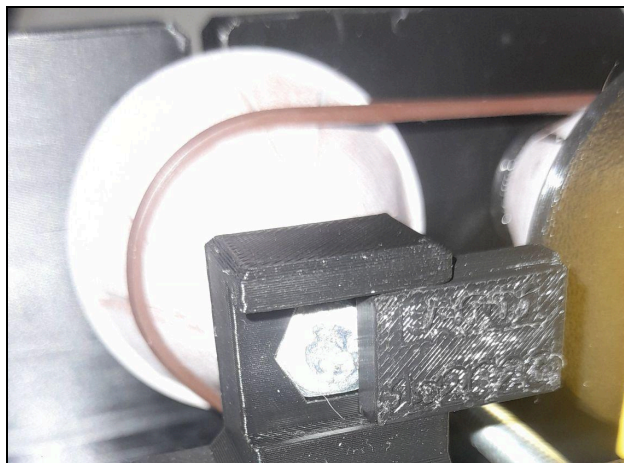


Figura 9 - Detalhe ponta de eixo removível.



Figura 10 - Roda de saída com eixo impresso em 3D.

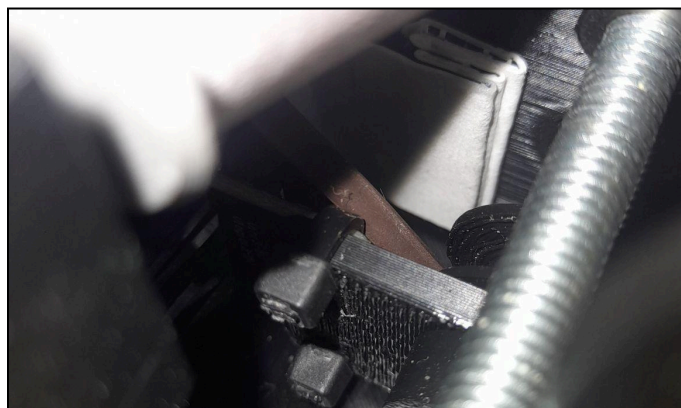


Figura 11 - Desvio para ter um fundo branco para o sensor ToF.



Figura 12 - Carregador de baterias 18650, 2 baterias em paralelo para os motores e uma para a BitDogLab.

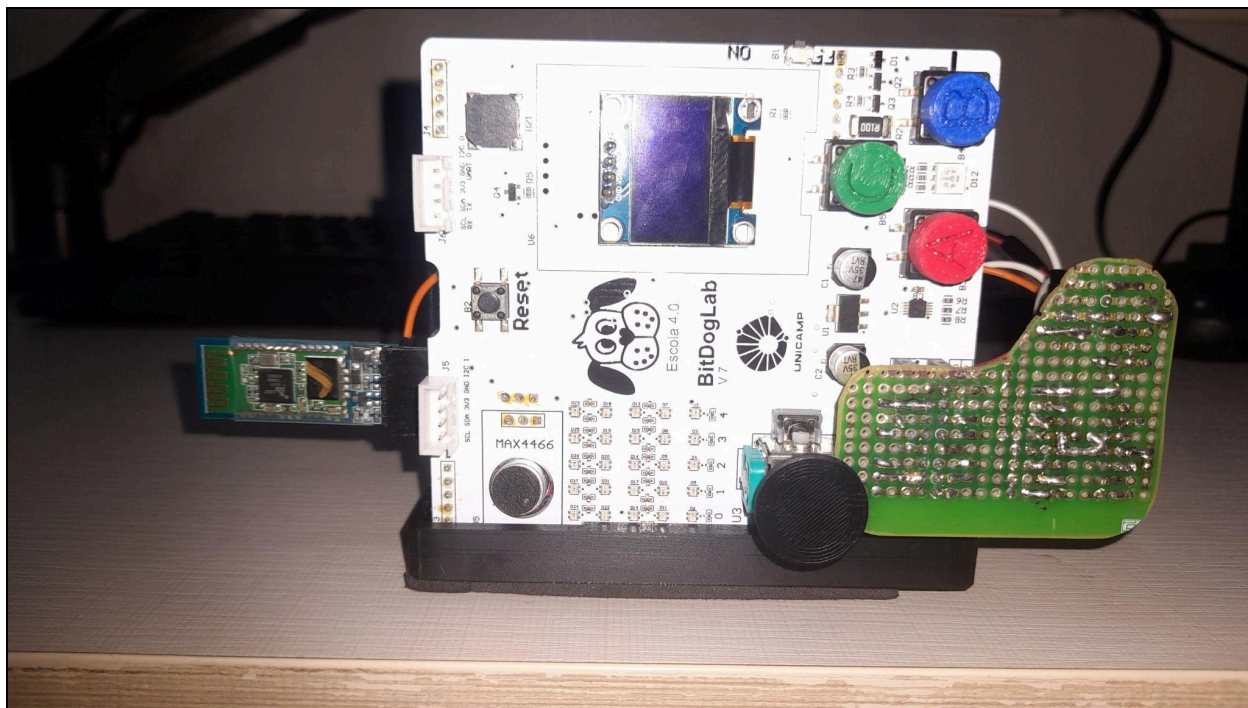


Figura 13 - Vista da placa.

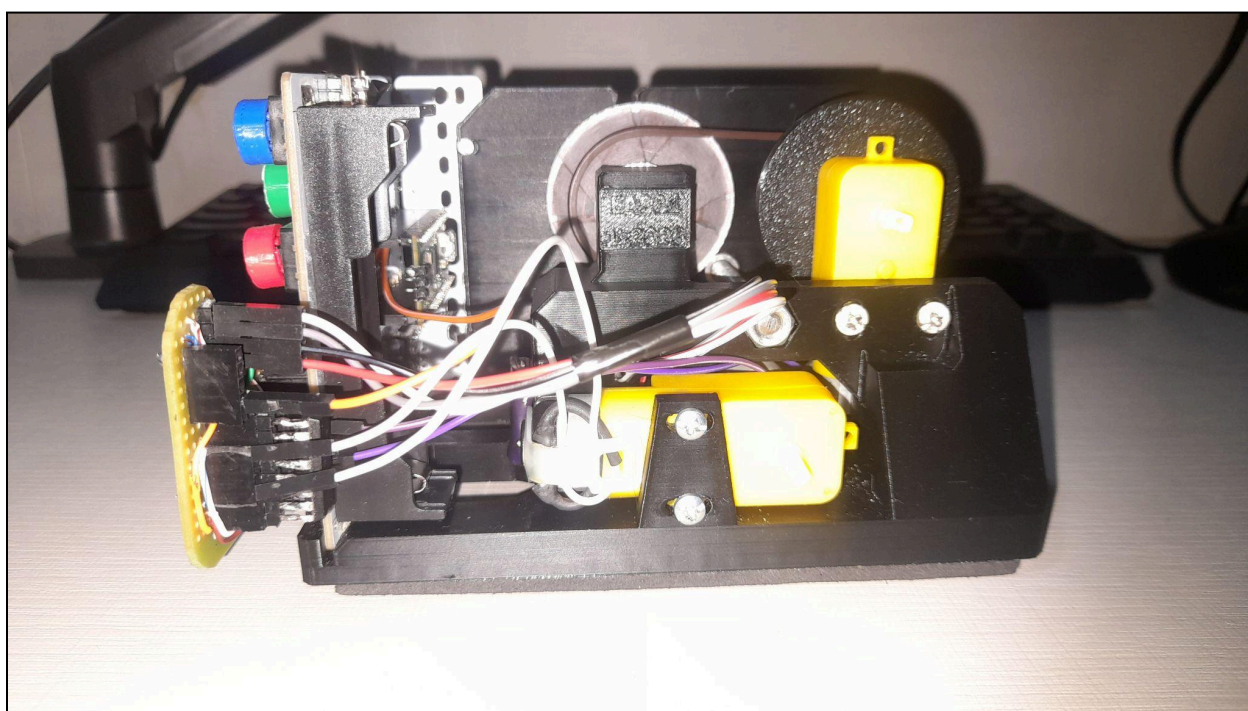


Figura 14 - Vista dos motores e polias (sem case das baterias dos motores).

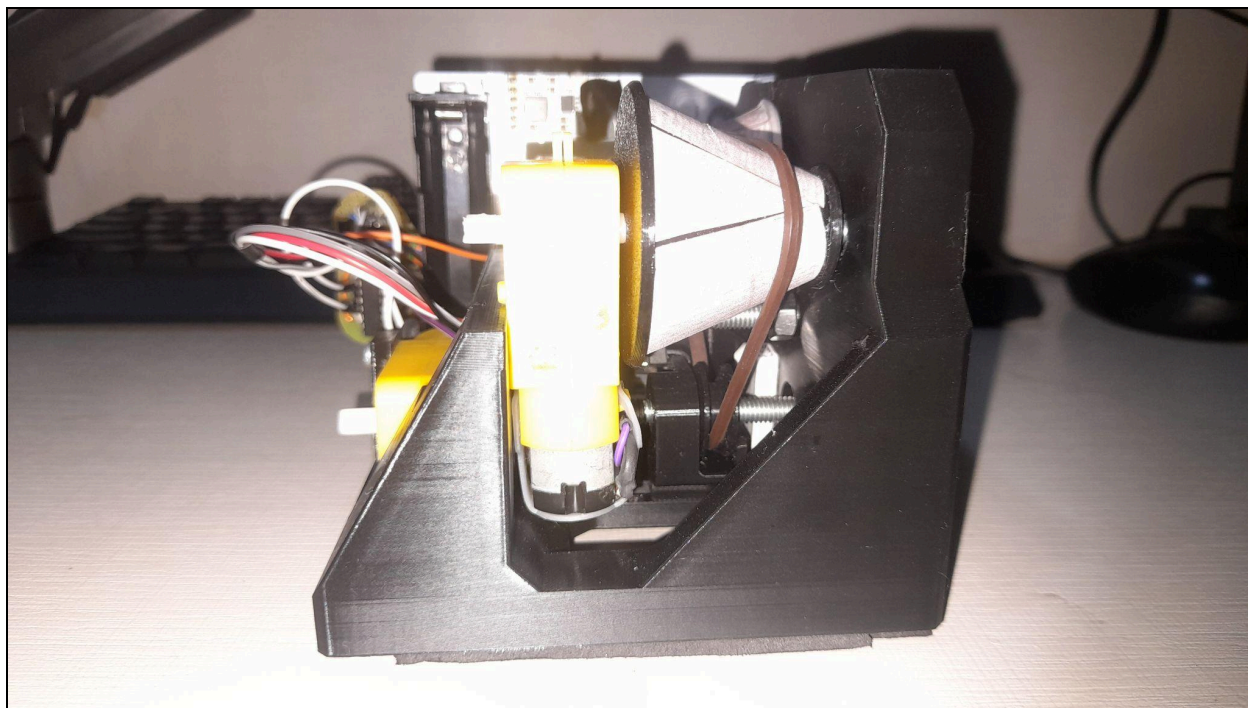


Figura 15 - Vista lateral, com detalhe do anel e da guia linear.

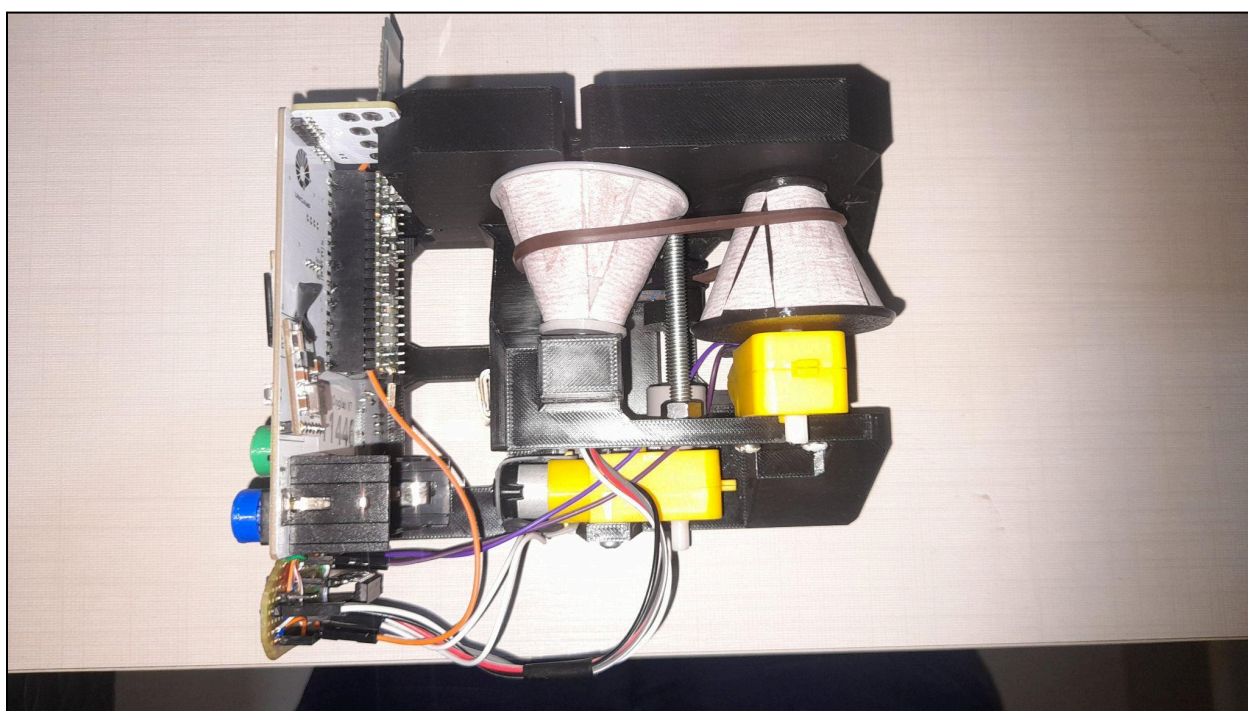


Figura 16 - Vista superior (sem case das baterias dos motores).

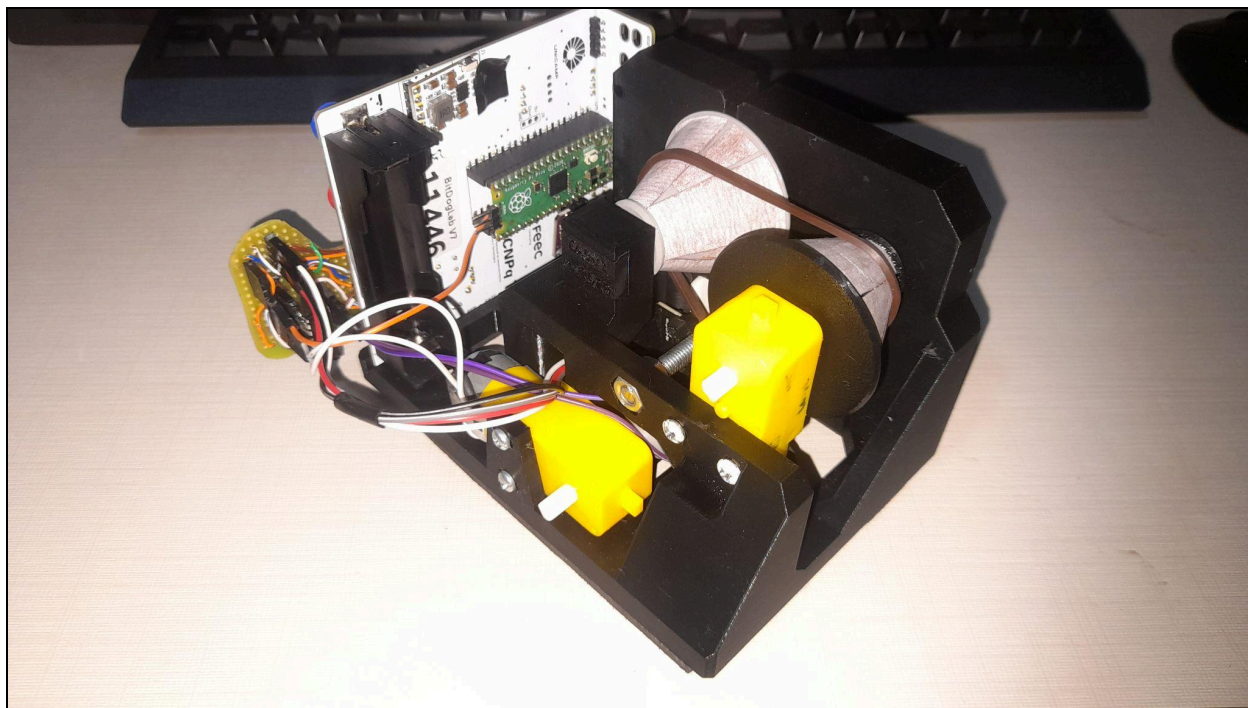


Figura 17 - Vista isométrica (sem case das baterias dos motores).

5. Testes realizados

A partir dos arquivos do projeto 2, era necessário implementar o controle via sensor ToF. Para isso, foi encontrada uma biblioteca no GitHub do sensor VL6180x para a placa ESP32.

A partir do código fonte desta biblioteca, e com o uso da ferramenta IA Gemini, foi criado o arquivo [vl6180x.py](#). A fim de afinar os parâmetros da biblioteca, foram criados outros dois códigos:

- Arquivo de calibração ([vl6180x_calibrate.py](#)): Disponibiliza um valor de offset para ser inputado no arquivo [vl6180x.py](#) (biblioteca).
- Arquivo de teste ([vl6180x_test.py](#)): Exibe no Terminal, periodicamente, o valor da leitura do sensor ToF.

Com estes arquivos, foi possível verificar que o sensor possui um range de detecção mínimo de 40 mm padrão (conforme datasheet) e que isso pode ser ajustado alterando o valor armazenado em um endereço de memória, tarefa esta realizada ao importar o arquivo [vl6180x.py](#) em [setup.py](#) (endereço de memória 0x024, SYSRANGE__PART_TO_PART_RANGE_OFFSET). Além disso, notou-se que a leitura

só era eficaz quando a superfície utilizada para reflexão do sinal infravermelho possui cores claras.

Com o sensor ToF montado na guia linear do câmbio, pedaços de lixas colados nos cones e sistema montado, os testes detectaram grande atrito que não permitia que o sistema girasse de maneira alguma. Por conta da situação de eixo travado, o consumo de corrente na ponte H L293D começou a afetar o funcionamento do motor do eixo.

Com isso, mais funções foram adicionadas ao sistema:

- Botão A: função de *Homing*, que busca trazer o câmbio para a posição inicial. Por se tratar de um sistema sem embreagem, é recomendado deixar o motor principal girando para não forçar os cones e o motor principal.
- Botão C: botão de emergência, que ao ser pressionado irá interromper qualquer tipo de acionamento dos motores ou funções específicas (e.g. *Homing*)

6. Conclusão

A realização deste projeto consolidou, de forma prática, os conceitos de integração entre hardware e software no desenvolvimento de sistemas mecatrônicos. O desenvolvimento da segunda versão do Câmbio de Transmissão Continuamente Variável (CVT) com a plataforma BitDogLab V7 permitiu uma exploração profunda dos desafios reais da engenharia, unindo projeto mecânico em CAD, eletrônica e programação de controle em malha fechada.

Durante a execução, diversos obstáculos físicos precisaram ser contornados para viabilizar o funcionamento mecânico. O alto atrito no giro e as limitações de aderência exigiram adaptações práticas, como a colagem de pedaços de lixa na superfície dos cones para garantir a tração da correia (anel de vedação). Além disso, a eficiência reflexiva do sensor *Time of Flight* (ToF) VL6180X foi inicialmente prejudicada pela cor preta do plástico impresso em 3D, problema este solucionado com um pedaço de papel branco, assegurando o retorno adequado do sinal infravermelho e leituras confiáveis de posicionamento.

Do lado da eletrônica e do software, o projeto exigiu grande robustez para lidar com a alta demanda de corrente e os ruídos provenientes do acionamento dos motores

via ponte H L293D, que chegaram a afetar a comunicação no barramento I2C. A implementação de um filtro de média móvel para os dados de distância, aliada aos tratamentos de exceção no driver do sensor para suportar falhas físicas instantâneas sem travar o código principal, garantiu que o firmware em MicroPython operasse de maneira assíncrona, fluida e ininterrupta.

Por fim, o sistema alcançou plenamente os objetivos propostos. A integração do aplicativo de celular operando via módulo Bluetooth HC-05 funcionou em perfeita sincronia com a IHM do display OLED, que exibiu a telemetria em tempo real das ações de controle. O trabalho evidenciou não apenas a viabilidade da BitDogLab V7 como unidade central de processamento, mas, principalmente, a importância da iteração contínua para refinar um projeto e superar as severas limitações físicas impostas pelo mundo real.

7. Referências

- **FRUETT, F.** *BitDogLab V7*: repositório oficial. Campinas: Unicamp, 2024. Disponível em: <https://gitlab.unicamp.br/fabiano/bitdoglab-v7>.
- **FRUETT, F.** *Banco de informações da BitDogLab V7*. [S. l.], 2024. Disponível em: https://docs.google.com/document/d/13-68OqiU7ISE8U2KPRUXT2ISeBI3WPhXjGD_FH52eWIU.
- **r223908; a213293.** *PJ2 EA801*: repositório de desenvolvimento do projeto. GitHub, 2026. Disponível em: https://github.com/r223908/pj2_ea801_r223908_a213293.
- **r223908; a213293.** *PJ3 EA801*: repositório de desenvolvimento do projeto. GitHub, 2026. Disponível em: https://github.com/r223908/pj3_ea801_r223908_a213293.
- **Ledbelly2142.** *VL6180X*: Micropython driver for VL6180X on ESP32. GitHub, 2026. Disponível em: <https://github.com/Ledbelly2142/VL6180X/blob/master/vl6180x.py>.
- **STMICROELECTRONICS.** *VL6180X*: Proximity and ambient light sensing (ALS) module. STMicroelectronics, [S. d.]. Disponível em: [VL6180X | Product - STMicroelectronics](#)